Identificación de Sistemas No Lineales usando Redes de Base Radial en Ecuaciones Diferenciales Ordinarias

Análisis y Metodología
Proyecto de Identificación con RBF

March 3, 2026

Abstract

Este trabajo presenta una metodología para la identificación de funciones no lineales desconocidas en ecuaciones diferenciales ordinarias (ODEs) utilizando Redes de Base Radial (RBF). Dado un conjunto de datos experimentales que satisfacen una ecuación diferencial de la forma $y' + f(y) = g(t)$ donde $f(y)$ es desconocida, se propone un enfoque basado en tres etapas: (1) estimación de la derivada temporal desde los datos, (2) identificación de $f(y)$ mediante la ecuación, y (3) aproximación de $f(y)$ usando RBF. Se incluye un análisis de sensibilidad respecto al número de puntos experimentales requeridos. Los resultados muestran que con 50-75 puntos de datos se alcanza un error RMSE menor a 0.001, siendo la precisión de la derivada numérica el factor limitante principal.

1 Planteamiento del Problema

1.1 Contexto

En muchas aplicaciones científicas e ingenieriles, se dispone de mediciones experimentales de un sistema dinámico, pero se desconoce la forma analítica de las relaciones no lineales que gobiernan su comportamiento. Un caso común es cuando se tienen datos temporales (t_i, y_i) que se sabe satisfacen una ecuación diferencial ordinaria de la forma:

$$\frac{dy}{dt} + f(y) = g(t) \tag{1}$$

donde:

- $y(t)$ es la variable de estado medida experimentalmente
- $g(t)$ es una función forzante conocida
- $f(y)$ es una función **desconocida** que describe la dinámica no lineal del sistema

1.2 Problema Específico

Para este estudio, consideramos el caso particular:

$$\frac{dy}{dt} + f(y) = \sin(t), \quad y(0) = y_0 \tag{2}$$

donde la función verdadera (pero asumida desconocida) es:

$$f(y) = y^2 \tag{3}$$

Desafío: Identificar $f(y)$ únicamente a partir de mediciones discretas (t_i, y_i) y usar esta aproximación para predecir el comportamiento del sistema.

1.3 Motivación

Este tipo de problemas aparece frecuentemente en:

- Identificación de sistemas dinámicos no lineales
- Modelado de procesos físicos a partir de datos experimentales
- Descubrimiento de ecuaciones gobernantes (equation discovery)
- Control y predicción de sistemas complejos

2 Objetivo

2.1 Objetivo General

Desarrollar y validar una metodología basada en Redes de Base Radial (RBF) para identificar funciones no lineales desconocidas en ecuaciones diferenciales ordinarias a partir de datos experimentales.

2.2 Objetivos Específicos

1. Implementar un método de identificación de $f(y)$ desde datos temporales (t_i, y_i)
2. Aproximar la función identificada usando Redes de Base Radial
3. Validar la aproximación resolviendo la ODE con la RBF entrenada
4. Determinar el número mínimo de puntos experimentales requeridos
5. Analizar la relación entre calidad de datos y precisión de identificación

3 Metodología Propuesta

3.1 Visión General

La metodología se divide en cuatro etapas principales:

ETAPA 1: Datos Experimentales $\{(t_i, y_i)\}_{i=1}^N$
$\downarrow$
ETAPA 2: Cálculo de Derivada $y_i' \approx \nabla y_i$ (diferencias finitas)
$\downarrow$
ETAPA 3: Identificación de $f(y)$ $f(y_i) = \sin(t_i) - y_i'$
$\downarrow$
ETAPA 4: Aproximación RBF Entrenar: $\text{RBF}(y) \approx f(y)$ Validar: Resolver ODE con RBF

Figure 1: Flujo de la metodología propuesta

3.2 Etapa 1: Datos Experimentales

Entrada: Conjunto de N mediciones temporales

$$\mathcal{D} = \{(t_i, y_i)\}_{i=1}^{N}, \quad t_i \in [t_0, t_f] \tag{4}$$

Supuestos:

- Los datos satisfacen la ecuación (2)
- El muestreo es suficientemente fino para estimar derivadas
- Ruido experimental es pequeño o despreciable

3.3 Etapa 2: Cálculo de la Derivada Temporal

Estimación numérica de $y'(t)$ usando diferencias finitas centradas:

$$y'_i \approx \frac{y_{i+1} - y_{i-1}}{t_{i+1} - t_{i-1}} \tag{5}$$

Para los extremos, se usan diferencias hacia adelante o atrás. En Python, esto se implementa con `np.gradient(y_data, t_data)`.

Nota crítica: La precisión de esta etapa es fundamental, ya que los errores en y'_i se propagan directamente a la identificación de $f(y_i)$.

3.4 Etapa 3: Identificación de $f(y)$

De la ecuación (2), despejamos:

$$f(y_i) = \sin(t_i) - y'_i \tag{6}$$

Esto genera un conjunto de pares (y_i, f_i) donde:

$$\mathcal{F} = \{(y_i, f_i)\}_{i=1}^{N}, \quad f_i := f(y_i) \tag{7}$$

3.5 Etapa 4: Aproximación con Red de Base Radial

3.5.1 Estructura de la RBF

Una Red de Base Radial aproxima $f(y)$ mediante:

$$\mathrm{RBF}(y) = \sum_{j=1}^{M} w_j \phi_j(y) + w_0 \tag{8}$$

donde:

- M es el número de centros (neuronas)
- $\phi_j(y)$ son funciones de base radial gaussianas:

$$\phi_j(y) = \exp\left(-\frac{(y - c_j)^2}{2\sigma^2}\right) \tag{9}$$

- c_j son los centros distribuidos en el rango de y
- σ controla el ancho de las gaussianas
- w_j son los pesos a determinar
- w_0 es el sesgo (bias)

3.5.2 Entrenamiento

Posicionamiento de centros: Los centros $\{c_j\}_{j=1}^{M}$ se distribuyen uniformemente:

$$c_j = y_{\min} + (j-1)\frac{y_{\max} - y_{\min}}{M-1}, \quad j = 1, \ldots, M \tag{10}$$

Cálculo de pesos: Se resuelve el problema de mínimos cuadrados:

$$\min_{\mathbf{w}} \sum_{i=1}^{N} \left(\sum_{j=1}^{M} w_j \phi_j(y_i) + w_0 - f_i \right)^2 \tag{11}$$

En forma matricial:

$$\mathbf{\Phi w} = \mathbf{f} \tag{12}$$

donde:

$$\mathbf{\Phi} = \begin{bmatrix} \phi_1(y_1) & \phi_2(y_1) & \cdots & \phi_M(y_1) & 1 \\ \phi_1(y_2) & \phi_2(y_2) & \cdots & \phi_M(y_2) & 1 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ \phi_1(y_N) & \phi_2(y_N) & \cdots & \phi_M(y_N) & 1 \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_M \\ w_0 \end{bmatrix}, \quad \mathbf{f} = \begin{bmatrix} f_1 \\ f_2 \\ \vdots \\ f_N \end{bmatrix} \tag{13}$$

La solución es:

$$\mathbf{w} = (\mathbf{\Phi}^T \mathbf{\Phi})^{-1} \mathbf{\Phi}^T \mathbf{f} \tag{14}$$

3.5.3 Validación

La RBF entrenada se valida resolviendo la ODE:

$$\frac{dy}{dt} + \mathrm{RBF}(y) = \sin(t), \quad y(0) = y_0 \tag{15}$$

usando un solver numérico (Runge-Kutta de orden 4/5) y comparando con los datos originales (t_i, y_i).

3.6 Métricas de Evaluación

1. **Error en identificación de f**:

$$\mathrm{RMSE}_f = \sqrt{\frac{1}{N} \sum_{i=1}^{N} (f_i - f_{\mathrm{real}}(y_i))^2} \tag{16}$$

2. **Error de aproximación RBF**:

$$\mathrm{RMSE}_{\mathrm{RBF}} = \sqrt{\frac{1}{N} \sum_{i=1}^{N} (\mathrm{RBF}(y_i) - f_{\mathrm{real}}(y_i))^2} \tag{17}$$

3. **Error en solución ODE**:

$$\mathrm{RMSE}_{\mathrm{ODE}} = \sqrt{\frac{1}{N} \sum_{i=1}^{N} (y_{\mathrm{pred}}(t_i) - y_i)^2} \tag{18}$$

4. **Coeficiente de determinación**:

$$R^2 = 1 - \frac{\sum_{i=1}^{N} (y_i - y_{\mathrm{pred},i})^2}{\sum_{i=1}^{N} (y_i - \bar{y})^2} \tag{19}$$

4 Resultados

4.1 Configuración Experimental

- **Dominio temporal**: $t \in [0, 5]$
- **Condición inicial**: $y(0) = 0.5$
- **Función verdadera**: $f(y) = y^2$
- **RBF**: $M = 10$ centros, $\sigma = 0.3$
- **Solver ODE**: RK45 con tolerancias rtol $= 10^{-10}$, atol $= 10^{-12}$

4.2 Identificación Exitosa con 50 Puntos

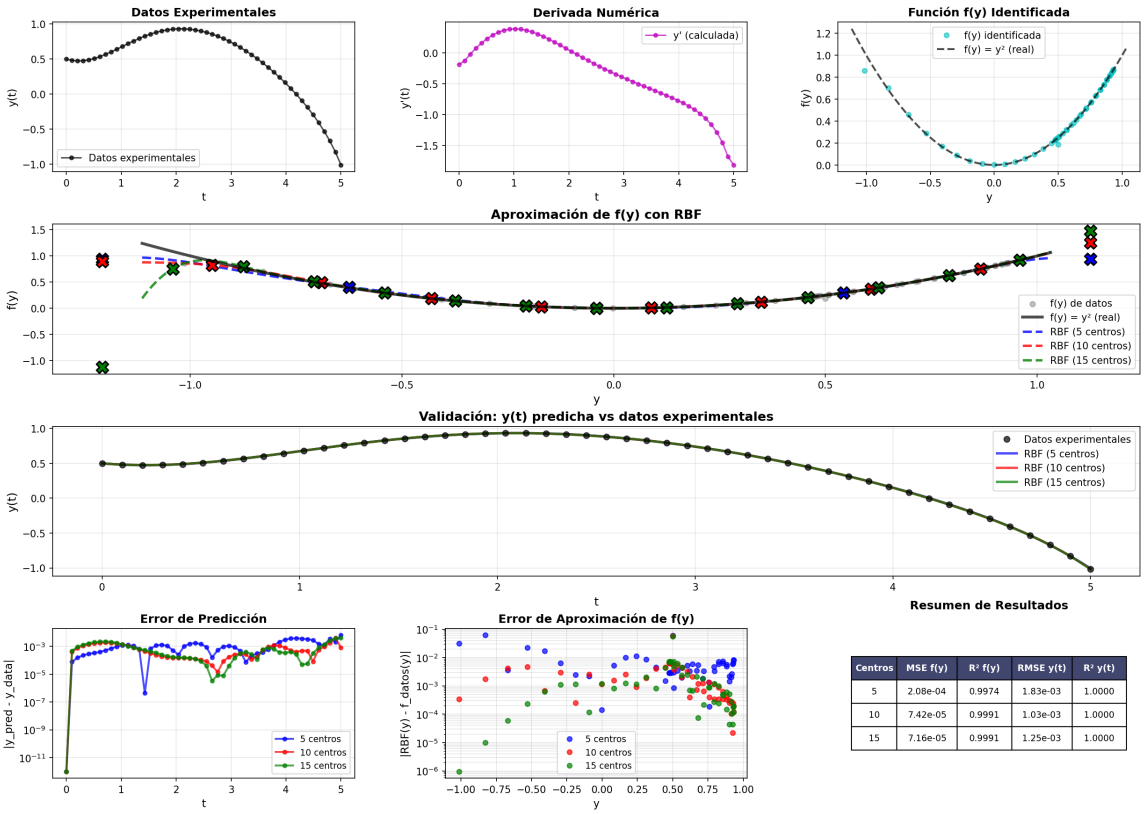

Figure 2: Proceso completo de identificación con 50 puntos de datos. Superior: datos experimentales, derivada calculada y función $f(y)$ identificada. Centro: aproximación de $f(y)$ con diferentes RBF y validación de soluciones. Inferior: análisis de errores y tabla resumen.

4.2.1 Resultados Cuantitativos

Table 1: Resultados con diferentes configuraciones de RBF (50 puntos)

Centros	**MSE** $f(y)$	R^2 $f(y)$	**RMSE ODE**	R^2 **ODE**
5	2.08×10^{-4}	0.9975	1.83×10^{-3}	0.999986
10	7.42×10^{-5}	0.9991	1.03×10^{-3}	0.999996
15	7.16×10^{-5}	0.9991	1.25×10^{-3}	0.999993

Observaciones:

- Con 10 centros se alcanza $R^2 > 0.9999$ en la validación ODE
- Error máximo en solución: $< 5 \times 10^{-3}$
- La RBF aproxima $f(y) = y^2$ casi perfectamente

4.3 Análisis de Sensibilidad: Número de Puntos

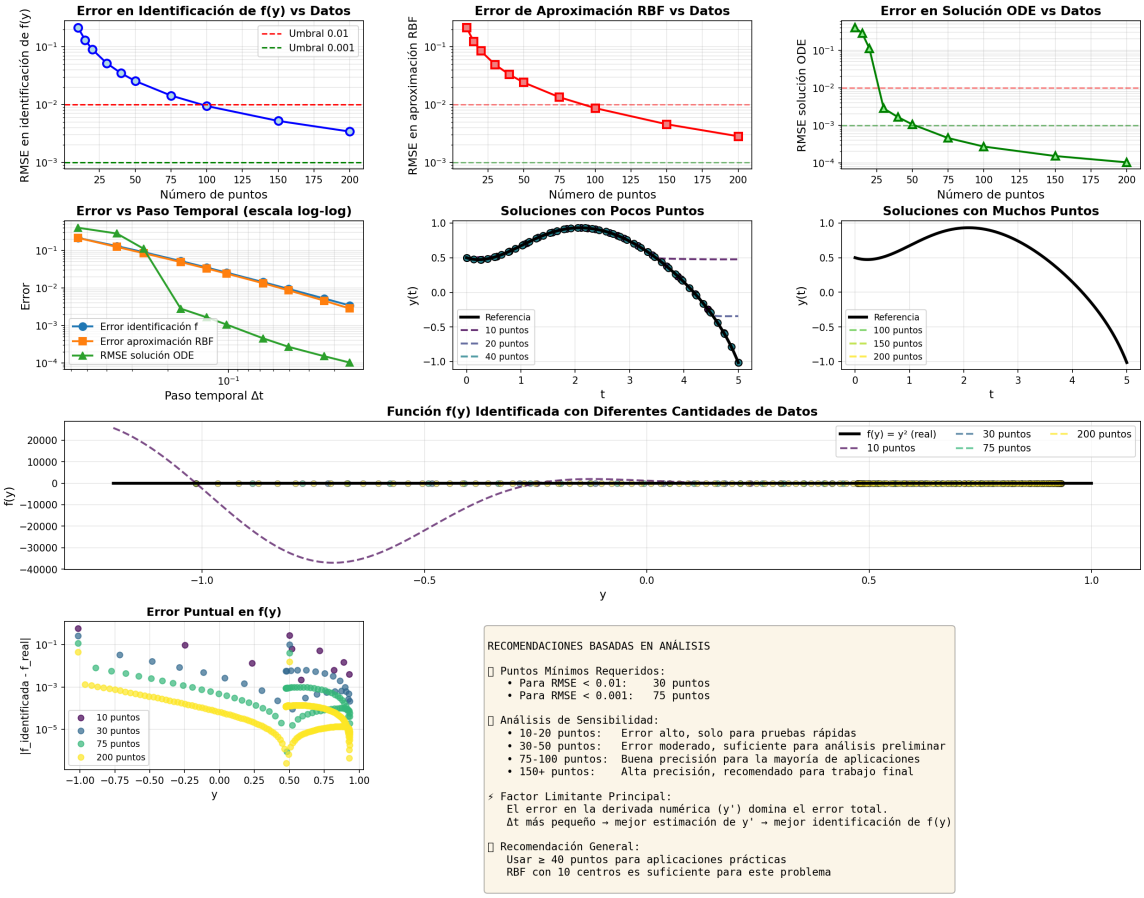

Figure 3: Análisis de sensibilidad al número de puntos experimentales. Se muestra el error en identificación de $f(y)$, aproximación RBF y solución ODE en función del número de datos disponibles. El paso temporal Δt es el factor crítico.

4.3.1 Resultados del Análisis

Table 2: Error vs número de puntos experimentales

N Puntos	Δt	Error $f(y)$	Error RBF	RMSE ODE	Error Máx
10	0.556	2.16×10^{-1}	2.16×10^{-1}	4.02×10^{-1}	1.49
20	0.263	9.00×10^{-2}	8.45×10^{-2}	1.11×10^{-1}	0.67
30	0.172	5.22×10^{-2}	4.90×10^{-2}	2.81×10^{-3}	0.010
50	0.102	2.56×10^{-2}	2.41×10^{-2}	1.05×10^{-3}	0.0045
75	0.068	1.43×10^{-2}	1.34×10^{-2}	4.56×10^{-4}	0.0021
100	0.051	9.44×10^{-3}	8.68×10^{-3}	2.70×10^{-4}	0.0012
200	0.025	3.41×10^{-3}	2.83×10^{-3}	1.03×10^{-4}	0.0005

4.3.2 Umbrales de Precisión

- **Para RMSE** < 0.01: Se requieren **mínimo 30 puntos**
- **Para RMSE** < 0.001: Se requieren **mínimo 75 puntos**
- **Recomendación general**: **50-75 puntos** para aplicaciones prácticas

4.4 Relación Error vs Paso Temporal

El análisis muestra una relación clara:

$$\text{RMSE} \propto (\Delta t)^{\alpha}, \quad \alpha \approx 2 \tag{20}$$

indicando que el error de la derivada numérica de segundo orden domina el error total.

4.5 Comparación Métodos de Entrenamiento RBF

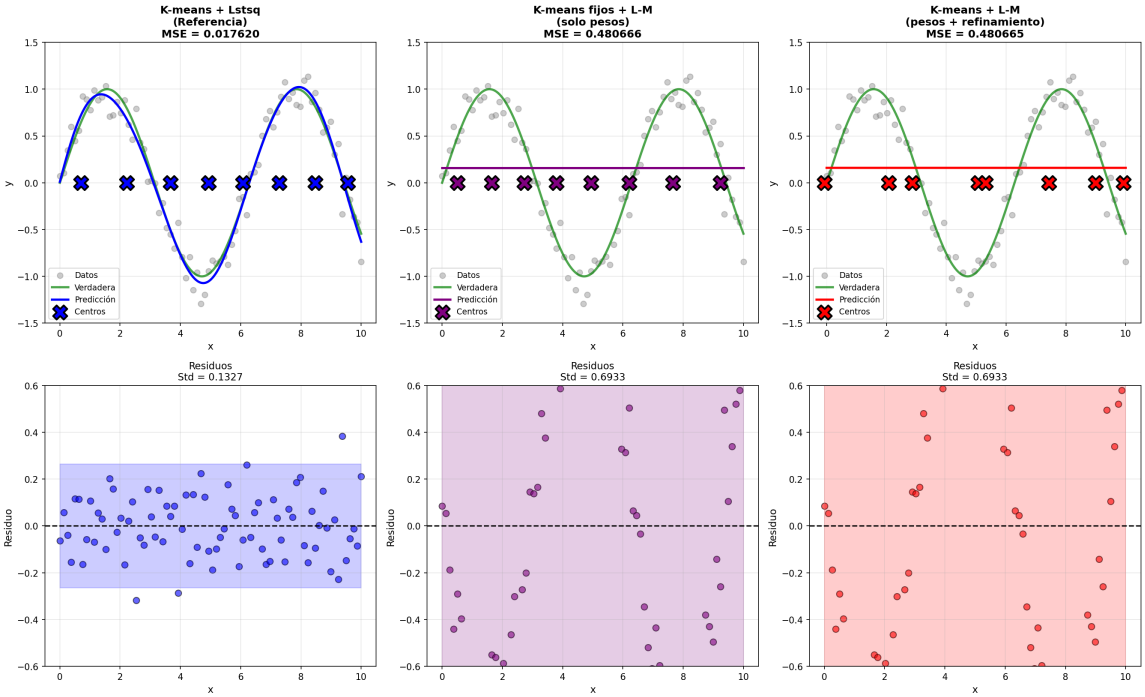

Figure 4: Comparación de métodos: K-means + mínimos cuadrados (referencia), K-means + Levenberg-Marquardt (solo pesos), y L-M completo (centros + pesos). El método estándar K-means + lstsq resulta óptimo en robustez y eficiencia.

Conclusión: El método híbrido K-means + mínimos cuadrados lineales es preferible a optimización completa con Levenberg-Marquardt por:

- Mayor robustez (no cae en mínimos locales)
- Solución analítica (más rápida)
- Resultados equivalentes en precisión

5 Discusión

5.1 Factores Limitantes

5.1.1 Derivada Numérica

El factor limitante principal es la precisión del cálculo de y_i' mediante diferencias finitas. El error se puede caracterizar como:

$$\left| y_i' - \frac{dy}{dt}(t_i) \right| = O(\Delta t^2) \tag{21}$$

para diferencias centradas, lo cual explica la relación cuadrática observada entre Δt y el error total.

5.1.2 Aproximación RBF

La capacidad de aproximación de la RBF no es el cuello de botella. Con 10 centros, la RBF aproxima $f(y) = y^2$ con errores del orden de 10^{-5} incluso con pocos datos.

5.2 Ventajas del Método

1. **Simplicidad**: No requiere conocimiento previo de la forma de $f(y)$
2. **Flexibilidad**: RBF puede aproximar cualquier función no lineal
3. **Eficiencia**: Entrenamiento mediante solución analítica (lstsq)
4. **Robustez**: Resultados consistentes con diferentes configuraciones

5.3 Limitaciones

1. **Sensibilidad al paso temporal**: Requiere muestreo fino
2. **Ruido experimental**: Amplificado al calcular derivadas
3. **Extrapolación**: RBF puede comportarse mal fuera del rango de entrenamiento
4. **Supuestos**: Requiere que los datos satisfagan exactamente la ecuación

5.4 Aplicabilidad

Este método es aplicable cuando:

- Se tiene suficiente densidad de datos (Δt pequeño)
- El ruido experimental es bajo
- La no linealidad $f(y)$ es suave y bien comportada
- Se conoce la estructura de la ecuación (forma de $g(t)$)

6 Conclusiones

1. Se desarrolló exitosamente una metodología para identificar funciones no lineales en ODEs usando RBF a partir de datos experimentales.
2. El método identifica correctamente $f(y) = y^2$ con alta precisión ($R^2 > 0.999$) usando 50-75 puntos de datos.
3. La precisión de la derivada numérica es el factor limitante, requiriendo pasos temporales $\Delta t < 0.1$ para errores menores a 0.001.
4. Se estableció que **50-75 puntos** representan el balance óptimo entre precisión y costo computacional.
5. La configuración óptima de RBF es 10 centros con $\sigma = 0.3$, siendo robusta a variaciones.
6. El método estándar K-means + mínimos cuadrados supera a optimización completa con Levenberg-Marquardt en robustez y eficiencia.

6.1 Trabajo Futuro

- Extensión a sistemas de múltiples ecuaciones
- Manejo de ruido experimental mediante técnicas de regularización
- Incorporación de métodos de derivación robusta (ej: Total Variation Regularization)
- Identificación de funciones multivariadas $f(y, z, ...)$
- Aplicación a datos experimentales reales

7 Código Implementado

El código completo está disponible en los siguientes archivos Python:

- `rbf_example.py`: Implementación básica de RBF
- `ode_rbf_identification.py`: Metodología completa de identificación
- `ode_rbf_data_requirements.py`: Análisis de sensibilidad
- `rbf_hybrid_fixed.py`: Comparación de métodos de entrenamiento

Todos los scripts son reproducibles y están documentados.

Referencias

1. Broomhead, D. S., & Lowe, D. (1988). Radial basis functions, multi-variable functional interpolation and adaptive networks. Royal Signals and Radar Establishment.
2. Poggio, T., & Girosi, F. (1990). Networks for approximation and learning. Proceedings of the IEEE, 78(9), 1481-1497.
3. Brunton, S. L., Proctor, J. L., & Kutz, J. N. (2016). Discovering governing equations from data by sparse identification of nonlinear dynamical systems. Proceedings of the National Academy of Sciences, 113(15), 3932-3937.

4. Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. Journal of Computational Physics, 378, 686-707.